Cosmos EVM 최적화 벤치마크 설계

개요

이 문서는 Phase 3 최적화 구현의 성능 개선을 검증하기 위한 종합적인 벤치마크 프레임워크를 설계한다. 실제 cosmos/evm 레포지토리의 기존 벤치마크 코드를 분석하고, 최적화 효과를 정량적으로 측정할 수 있는 확장된 벤치마크 스위트를 제안한다.

분석 대상 코드:

- `tests/integration/x/vm/benchmark_statedb.go` - StateDB 연산 벤치마크
- `tests/integration/x/vm/state_transition_benchmark.go` - 트랜잭션 처리 벤치마크
- `tests/integration/x/vm/test_benchmark.go` - 스마트 컨트랙트 벤치마크
- `x/vm/store/snapshotmulti/store_bench_test.go` - 스토어 벤치마크

---

1. 기존 벤치마크 분석

1.1 StateDB 벤치마크 (`benchmark_statedb.go`)

현재 cosmos/evm에는 StateDB 연산에 대한 마이크로 벤치마크가 구현되어 있다:

```
// tests/integration/x/vm/benchmark_statedb.go:16-30
func BenchmarkCreateAccountNew(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()
    vmdb := suite.StateDB()

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        b.StopTimer()
        addr := utiltx.GenerateAddress()
        b.StartTimer()
        vmdb.CreateAccount(addr)
    }
}
```

기존 벤치마크 함수들:

함수명	측정 대상	특징
`BenchmarkCreateAccountNew`	신규 계정 생성	주소 생성은 시간 측정 제외
`BenchmarkCreateAccountExisting`	기존 계정 재생성	같은 주소 반복
`BenchmarkAddBalance`	잔액 추가	uint256 연산 포함
`BenchmarkSetCode`	코드 저장	바이트코드 해시 연산
`BenchmarkSetState`	상태 저장	Storage slot 쓰기
`BenchmarkAddLog`	로그 추가	이벤트 기록
`BenchmarkSnapshot`	스냅샷 생성	Journal 체크포인트
`BenchmarkSubBalance`	잔액 차감	언더플로우 체크 포함
`BenchmarkSetNonce`	논스 설정	시퀀스 업데이트
`BenchmarkAddRefund`	리펀드 추가	Gas 환불 누적
`BenchmarkSuicide`	셀프 디스트럭트	계정 삭제

1.2 State Transition 벤치마크 (`state_transition_benchmark.go`)

트랜잭션 유형별 처리 성능을 측정한다:

```go
// tests/integration/x/vm/state_transition_benchmark.go:178-205
func BenchmarkApplyTransaction(b *testing.B) {
    suite := KeeperTestSuite{EnableLondonHF: true}
    suite.SetupTest()

    ethSigner := ethtypes.LatestSignerForChainID(evmtypes.GetEthChainConfig().ChainID)

    b.ResetTimer()
    b.ReportAllocs()
    for i := 0; i < b.N; i++ {
        b.StopTimer()
        addr := suite.Keyring.GetAddr(0)
        krSigner := utiltx.NewSigner(suite.Keyring.GetPrivKey(0))
        tx, err := newSignedEthTx(templateAccessListTx,
            suite.Network.App.GetEVMKeeper().GetNonce(suite.Network.GetContext(), addr),
            sdk.AccAddress(addr.Bytes()),
            krSigner,
            ethSigner,
        )
        require.NoError(b, err)

        b.StartTimer()
        resp, err := suite.Network.App.GetEVMKeeper().ApplyTransaction(
            suite.Network.GetContext(), tx.AsTransaction())
        b.StopTimer()

        require.NoError(b, err)
        require.False(b, resp.Failed())
    }
}
```

트랜잭션 유형별 벤치마크:

함수명	트랜잭션 유형	측정 범위
`BenchmarkApplyTransaction`	AccessList	전체 트랜잭션 처리
`BenchmarkApplyTransactionWithLegacyTx`	Legacy	EIP-155 이전 형식
`BenchmarkApplyTransactionWithDynamicFeeTx`	EIP-1559	동적 가스비
`BenchmarkApplyMessage`	AccessList	메시지만 처리
`BenchmarkApplyMessageWithLegacyTx`	Legacy	메시지만 처리
`BenchmarkApplyMessageWithDynamicFeeTx`	EIP-1559	메시지만 처리

1.3 스마트 컨트랙트 벤치마크 (`test_benchmark.go`)

실제 ERC20 컨트랙트 호출 성능을 측정한다:

```go
// tests/integration/x/vm/test_benchmark.go:85-104
func BenchmarkTokenTransfer(b *testing.B) {
    erc20Contract, err := testdata.LoadERC20Contract()
    require.NoError(b, err, "failed to load erc20 contract")

    DoBenchmark(b, func(suite *KeeperTestSuite, contract common.Address) *types.MsgEthereumTx {
        input, err := erc20Contract.ABI.Pack("transfer",
            common.HexToAddress("0x378c50D9264C63F3F92B806d4ee56E9D86FfB3Ec"),
            big.NewInt(1000))
        require.NoError(b, err)
        nonce := suite.Network.App.GetEVMKeeper().GetNonce(
            suite.Network.GetContext(), suite.Keyring.GetAddr(0))
        ethTxParams := &types.EvmTxArgs{
            ChainID:  types.GetEthChainConfig().ChainID,
            Nonce:    nonce,
            To:       &contract,
            Amount:   big.NewInt(0),
            GasLimit: 410000,
            GasPrice: big.NewInt(1),
            Input:    input,
        }
        return types.NewTx(ethTxParams)
    })
}
```

컨트랙트 벤치마크 함수들:

함수명	측정 대상	특징
`BenchmarkTokenTransfer`	ERC20 transfer	2개 Storage 업데이트
`BenchmarkEmitLogs`	대량 로그 발생	1000개 이벤트
`BenchmarkTokenTransferFrom`	ERC20 transferFrom	3개 Storage 업데이트
`BenchmarkTokenMint`	ERC20 mint	공급량 + 잔액 업데이트
`BenchmarkMessageCall`	내부 호출	10000회 반복 호출

1.4 스토어 벤치마크 (`store_bench_test.go`)

캐시 멀티스토어의 순차 처리 성능을 측정한다:

```go
// x/vm/store/snapshotmulti/store_bench_test.go:45-66
func benchmarkSequential(b *testing.B) {
    b.Helper()
    cms, keys := setupCacheMultiStoreWithKeys(20, 200000)
    selected := keys[:5]
    txs := *benchTxCount

    b.ReportAllocs()
    b.ResetTimer()
    for i := 0; i < b.N; i++ {
        for tx := 0; tx < txs; tx++ {
            for j := 0; j < 10; j++ {
                for _, key := range selected {
                    kv := cms.GetKVStore(key)
                    kv.Get([]byte(fmt.Sprintf("%s-key-%d", key.Name(), j%20)))
                    kv.Set([]byte(fmt.Sprintf("%s-tx-%d-%d", key.Name(), tx, j)))
                }
                snapshot := cms.Snapshot()
                benchSink = snapshot
            }
        }
    }
}
```

벤치마크 파라미터:

- 스토어 수: 20개
- 초기 엔트리 수: 200,000개
- 선택된 스토어: 5개
- 트랜잭션 수: 256개 (기본값)
- 연산 횟수/트랜잭션: 10회

---

2. 최적화별 벤치마크 설계

2.1 Storage Read Cache 벤치마크

목표: 블록 레벨 읽기 캐시의 효과 측정

2.1.1 캐시 히트율 벤치마크

```go
// benchmark_read_cache.go
package vm

import (
    "testing"
    "github.com/ethereum/go-ethereum/common"
)

// BenchmarkStorageReadCacheHit measures read performance with cache hits
func BenchmarkStorageReadCacheHit(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()

    // Setup: Create contract with storage
    contractAddr := deployTestContract(suite)

    // Pre-populate cache by reading once
    vmdb := suite.StateDB()
    slot := common.HexToHash("0x01")
    _ = vmdb.GetState(contractAddr, slot)

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        _ = vmdb.GetState(contractAddr, slot) // Should hit cache
    }
}

// BenchmarkStorageReadCacheMiss measures read performance with cache misses
func BenchmarkStorageReadCacheMiss(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()

    contractAddr := deployTestContract(suite)
    vmdb := suite.StateDB()

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        b.StopTimer()
        slot := common.BigToHash(big.NewInt(int64(i)))
        b.StartTimer()
        _ = vmdb.GetState(contractAddr, slot) // Cache miss
    }
}

// BenchmarkStorageReadCacheMixed measures mixed hit/miss pattern
func BenchmarkStorageReadCacheMixed(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()

    contractAddr := deployTestContract(suite)
    vmdb := suite.StateDB()

    // Pre-populate some slots
    for i := 0; i < 100; i++ {
        slot := common.BigToHash(big.NewInt(int64(i)))
        _ = vmdb.GetState(contractAddr, slot)
    }

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        // 80% hit, 20% miss pattern
        slotIdx := i % 125
        slot := common.BigToHash(big.NewInt(int64(slotIdx)))
        _ = vmdb.GetState(contractAddr, slot)
    }
}
```

2.1.2 블록 레벨 캐시 공유 벤치마크

```go
// BenchmarkBlockLevelCacheSharing measures cache effectiveness across transactions
func BenchmarkBlockLevelCacheSharing(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()

    contractAddr := deployTestContract(suite)

    // 블록 내 여러 트랜잭션 시뮬레이션
    b.Run("WithoutBlockCache", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            // 각 트랜잭션마다 새 StateDB
            vmdb := suite.StateDB()
            for j := 0; j < 10; j++ {
                slot := common.BigToHash(big.NewInt(int64(j)))
                _ = vmdb.GetState(contractAddr, slot)
            }
            vmdb.Commit()
        }
    })

    b.Run("WithBlockCache", func(b *testing.B) {
        blockCache := NewBlockReadCache(1000)

        for i := 0; i < b.N; i++ {
            vmdb := suite.StateDBWithCache(blockCache)
            for j := 0; j < 10; j++ {
                slot := common.BigToHash(big.NewInt(int64(j)))
                _ = vmdb.GetState(contractAddr, slot)
            }
            vmdb.Commit()
        }
    })
}
```

2.1.3 Bloom Filter 효과 벤치마크

```go
// BenchmarkBloomFilterEffectiveness measures Bloom filter false positive impact
func BenchmarkBloomFilterEffectiveness(b *testing.B) {
    cache := NewBlockReadCache(10000)

    // 다양한 엔트리 수로 테스트
    entryCounts := []int{100, 1000, 10000}

    for _, count := range entryCounts {
        b.Run(fmt.Sprintf("Entries_%d", count), func(b *testing.B) {
            // Populate cache
            for i := 0; i < count; i++ {
                addr := common.BigToAddress(big.NewInt(int64(i)))
                slot := common.BigToHash(big.NewInt(int64(i)))
                cache.Set(addr, slot, common.BigToHash(big.NewInt(int64(i*100))))
            }

            falsePositives := 0
            totalLookups := 0

            b.ResetTimer()
            for i := 0; i < b.N; i++ {
                // 존재하지 않는 키 조회
                addr := common.BigToAddress(big.NewInt(int64(count + i)))
                slot := common.BigToHash(big.NewInt(int64(i)))

                if cache.MayContain(addr, slot) {
                    totalLookups++
                    if _, found := cache.Get(addr, slot); !found {
                        falsePositives++
                    }
                }
            }

            b.ReportMetric(float64(falsePositives)/float64(b.N)*100, "false_positive_%")
        })
    }
}
```

2.2 Batched State Commits 벤치마크

목표: 배치 커밋의 효과 측정

2.2.1 개별 vs 배치 커밋 비교

```go
// benchmark_batch_commit.go
package vm

import (
    "testing"
)

// BenchmarkIndividualCommits measures individual commit overhead
func BenchmarkIndividualCommits(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()
    vmdb := suite.StateDB()

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        // 각 상태 변경 후 즉시 커밋 (현재 방식)
        addr := common.BigToAddress(big.NewInt(int64(i)))
        vmdb.CreateAccount(addr)
        vmdb.SetNonce(addr, uint64(i), tracing.NonceChangeUnspecified)
        vmdb.AddBalance(addr, uint256.NewInt(1000), tracing.BalanceChangeUnspecified)

        // 개별적으로 KVStore에 쓰기
        vmdb.Commit()
    }
}

// BenchmarkBatchedCommits measures batched commit performance
func BenchmarkBatchedCommits(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()
    vmdb := suite.StateDBWithBatching()

    batchSize := 100

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        addr := common.BigToAddress(big.NewInt(int64(i)))
        vmdb.CreateAccount(addr)
        vmdb.SetNonce(addr, uint64(i), tracing.NonceChangeUnspecified)
        vmdb.AddBalance(addr, uint256.NewInt(1000), tracing.BalanceChangeUnspecified)

        // batchSize마다 한 번 플러시
        if i % batchSize == batchSize - 1 {
            vmdb.FlushBatch()
        }
    }
}
```

2.2.2 배치 크기별 성능 벤치마크

```go
// BenchmarkBatchSizeImpact measures performance across different batch sizes
func BenchmarkBatchSizeImpact(b *testing.B) {
    batchSizes := []int{1, 10, 50, 100, 500, 1000}

    for _, size := range batchSizes {
        b.Run(fmt.Sprintf("BatchSize_%d", size), func(b *testing.B) {
            suite := KeeperTestSuite{}
            suite.SetupTest()
            vmdb := suite.StateDBWithBatching()

            b.ResetTimer()
            b.ReportAllocs()

            changes := 0
            for i := 0; i < b.N; i++ {
                addr := common.BigToAddress(big.NewInt(int64(i)))
                slot := common.BigToHash(big.NewInt(int64(i)))
                value := common.BigToHash(big.NewInt(int64(i * 100)))

                vmdb.SetState(addr, slot, value)
                changes++

                if changes >= size {
                    vmdb.FlushBatch()
                    changes = 0
                }
            }

            if changes > 0 {
                vmdb.FlushBatch()
            }
        })
    }
}
```

2.2.3 메모리 사용량 벤치마크

```go
// BenchmarkBatchMemoryUsage measures memory overhead of batching
func BenchmarkBatchMemoryUsage(b *testing.B) {
    batchSizes := []int{100, 500, 1000, 5000}

    for _, size := range batchSizes {
        b.Run(fmt.Sprintf("BatchSize_%d", size), func(b *testing.B) {
            var m runtime.MemStats
            runtime.GC()
            runtime.ReadMemStats(&m)
            startAlloc := m.TotalAlloc

            suite := KeeperTestSuite{}
            suite.SetupTest()
            writer := NewBatchWriter(suite.Context(), suite.Keeper())

            // Accumulate batch entries
            for i := 0; i < size; i++ {
                addr := common.BigToAddress(big.NewInt(int64(i)))
                slot := common.BigToHash(big.NewInt(int64(i)))
                value := common.BigToHash(big.NewInt(int64(i * 100)))
                writer.SetState(addr, slot, value)
            }

            runtime.GC()
            runtime.ReadMemStats(&m)

            b.ReportMetric(float64(m.TotalAlloc-startAlloc)/float64(size), "bytes/entry")
            b.ReportMetric(float64(m.TotalAlloc-startAlloc)/1024/1024, "MB_total")
        })
    }
}
```

2.3 Journal Optimization 벤치마크

목표: Journal 사전 할당 및 풀링 효과 측정

2.3.1 Journal 할당 벤치마크

```go
// benchmark_journal.go
package vm

import (
    "testing"
)

// BenchmarkJournalAppendDynamic measures dynamic slice expansion
func BenchmarkJournalAppendDynamic(b *testing.B) {
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        journal := newJournal() // 기존: 동적 확장

        for j := 0; j < 1000; j++ {
            journal.append(&storageChange{
                account: common.BigToAddress(big.NewInt(int64(j))),
                key:     common.BigToHash(big.NewInt(int64(j))),
                prev:    common.BigToHash(big.NewInt(0)),
            })
        }
    }
}

// BenchmarkJournalAppendPreallocated measures pre-allocated journal
func BenchmarkJournalAppendPreallocated(b *testing.B) {
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        journal := newJournalWithCapacity(1024) // 개선: 사전 할당

        for j := 0; j < 1000; j++ {
            journal.append(&storageChange{
                account: common.BigToAddress(big.NewInt(int64(j))),
                key:     common.BigToHash(big.NewInt(int64(j))),
                prev:    common.BigToHash(big.NewInt(0)),
            })
        }
    }
}
```

2.3.2 Object Pool 효과 벤치마크

```go
// BenchmarkJournalEntryAllocation measures entry allocation overhead
func BenchmarkJournalEntryAllocation(b *testing.B) {
    b.Run("NewAllocation", func(b *testing.B) {
        b.ReportAllocs()

        for i := 0; i < b.N; i++ {
            entry := &storageChange{
                account: common.BigToAddress(big.NewInt(int64(i))),
                key:     common.BigToHash(big.NewInt(int64(i))),
                prev:    common.BigToHash(big.NewInt(0)),
            }
            _ = entry
        }
    })

    b.Run("PooledAllocation", func(b *testing.B) {
        pool := &sync.Pool{
            New: func() interface{} {
                return &storageChange{}
            },
        }

        b.ReportAllocs()

        for i := 0; i < b.N; i++ {
            entry := pool.Get().(*storageChange)
            entry.account = common.BigToAddress(big.NewInt(int64(i)))
            entry.key = common.BigToHash(big.NewInt(int64(i)))
            entry.prev = common.BigToHash(big.NewInt(0))

            pool.Put(entry)
        }
    })
}
```

2.3.3 Snapshot/Revert 벤치마크

```go
// BenchmarkSnapshotRevert measures snapshot and revert performance
func BenchmarkSnapshotRevert(b *testing.B) {
    suite := KeeperTestSuite{}
    suite.SetupTest()
    vmdb := suite.StateDB()

    addr := suite.Keyring.GetAddr(0)

    b.Run("ShallowRevert", func(b *testing.B) {
        b.ReportAllocs()

        for i := 0; i < b.N; i++ {
            snapshot := vmdb.Snapshot()

            // 소량의 변경
            for j := 0; j < 10; j++ {
                slot := common.BigToHash(big.NewInt(int64(j)))
                vmdb.SetState(addr, slot, common.BigToHash(big.NewInt(int64(i*10+j))))
            }

            vmdb.RevertToSnapshot(snapshot)
        }
    })

    b.Run("DeepRevert", func(b *testing.B) {
        b.ReportAllocs()

        for i := 0; i < b.N; i++ {
            snapshot := vmdb.Snapshot()

            // 대량의 변경
            for j := 0; j < 1000; j++ {
                slot := common.BigToHash(big.NewInt(int64(j)))
                vmdb.SetState(addr, slot, common.BigToHash(big.NewInt(int64(i*1000+j))))
            }

            vmdb.RevertToSnapshot(snapshot)
        }
    })

    b.Run("NestedSnapshots", func(b *testing.B) {
        b.ReportAllocs()

        for i := 0; i < b.N; i++ {
            snapshots := make([]int, 10)

            for depth := 0; depth < 10; depth++ {
                snapshots[depth] = vmdb.Snapshot()

                for j := 0; j < 10; j++ {
                    slot := common.BigToHash(big.NewInt(int64(depth*10+j)))
                    vmdb.SetState(addr, slot, common.BigToHash(big.NewInt(int64(i))))
                }
            }

            // Revert all nested snapshots
            for depth := 9; depth >= 0; depth-- {
                vmdb.RevertToSnapshot(snapshots[depth])
            }
        }
    })
}
```

---

3. 통합 벤치마크 시나리오

3.1 DeFi 시뮬레이션 벤치마크

실제 DeFi 프로토콜의 사용 패턴을 시뮬레이션한다.

```go
// benchmark_defi_simulation.go
package vm

import (
    "testing"
    "math/big"
)

// BenchmarkUniswapSwap simulates Uniswap-style token swap
func BenchmarkUniswapSwap(b *testing.B) {
    suite, poolContract := SetupUniswapPool(b)

    krSigner := utiltx.NewSigner(suite.Keyring.GetPrivKey(0))
    ethCfg := types.GetEthChainConfig()

    b.ResetTimer()
    b.ReportAllocs()

    for i := 0; i < b.N; i++ {
        b.StopTimer()

        // swap(amountIn, amountOutMin, path, to, deadline)
        input, _ := uniswapABI.Pack("swapExactTokensForTokens",
            big.NewInt(1000000),
            big.NewInt(900000),
            []common.Address{tokenA, tokenB},
            suite.Keyring.GetAddr(0),
            big.NewInt(time.Now().Unix() + 3600),
        )

        nonce := suite.Network.App.GetEVMKeeper().GetNonce(
            suite.Network.GetContext(), suite.Keyring.GetAddr(0))

        msg := types.NewTx(&types.EvmTxArgs{
            ChainID:  ethCfg.ChainID,
            Nonce:    nonce,
            To:       &poolContract,
            Amount:   big.NewInt(0),
            GasLimit: 300000,
            GasPrice: big.NewInt(1),
            Input:    input,
        })
        msg.From = suite.Keyring.GetAddr(0).Bytes()
        msg.Sign(ethtypes.LatestSignerForChainID(ethCfg.ChainID), krSigner)

        b.StartTimer()

        ctx, _ := suite.Network.GetContext().CacheContext()
        rsp, err := suite.Network.App.GetEVMKeeper().EthereumTx(ctx, msg)

        b.StopTimer()

        require.NoError(b, err)
        require.False(b, rsp.Failed())
    }
}

// BenchmarkAaveLendingCycle simulates Aave-style lending operations
func BenchmarkAaveLendingCycle(b *testing.B) {
    suite, lendingPool := SetupAaveLendingPool(b)

    b.Run("Deposit", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            // deposit(asset, amount, onBehalfOf, referralCode)
            executeAaveOperation(b, suite, lendingPool, "deposit",
                tokenAddr, big.NewInt(1000000), suite.Keyring.GetAddr(0), uint16(0))
        }
    })

    b.Run("Borrow", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            // borrow(asset, amount, interestRateMode, referralCode, onBehalfOf)
            executeAaveOperation(b, suite, lendingPool, "borrow",
                tokenAddr, big.NewInt(500000), uint256.NewInt(2), uint16(0),
                suite.Keyring.GetAddr(0))
        }
    })

    b.Run("Repay", func(b *testing.B) {
```

```go
        for i := 0; i < b.N; i++ {
            // repay(asset, amount, rateMode, onBehalfOf)
            executeAaveOperation(b, suite, lendingPool, "repay",
                tokenAddr, big.NewInt(500000), uint256.NewInt(2),
                suite.Keyring.GetAddr(0))
        }
    })

    b.Run("Withdraw", func(b *testing.B) {
        for i := 0; i < b.N; i++ {
            // withdraw(asset, amount, to)
            executeAaveOperation(b, suite, lendingPool, "withdraw",
                tokenAddr, big.NewInt(1000000), suite.Keyring.GetAddr(0))
        }
    })
}
```

3.2 블록 빌드 시뮬레이션 벤치마크

```go
// BenchmarkBlockBuilding simulates full block building with multiple transactions
func BenchmarkBlockBuilding(b *testing.B) {
    txCounts := []int{10, 50, 100, 500}

    for _, count := range txCounts {
        b.Run(fmt.Sprintf("TxCount_%d", count), func(b *testing.B) {
            suite := KeeperTestSuite{EnableLondonHF: true}
            suite.SetupTest()

            // Prepare transactions
            txs := make([]*types.MsgEthereumTx, count)
            for i := 0; i < count; i++ {
                txs[i] = prepareSimpleTransferTx(suite, uint64(i))
            }

            b.ResetTimer()
            b.ReportAllocs()

            for i := 0; i < b.N; i++ {
                ctx, _ := suite.Network.GetContext().CacheContext()

                // BeginBlock
                suite.Network.App.GetEVMKeeper().BeginBlock(ctx)

                // Process all transactions
                for _, tx := range txs {
                    _, err := suite.Network.App.GetEVMKeeper().ApplyTransaction(
                        ctx, tx.AsTransaction())
                    require.NoError(b, err)
                }

                // EndBlock
                suite.Network.App.GetEVMKeeper().EndBlock(ctx)
            }
        })
    }
}
```

3.3 상태 크기별 성능 벤치마크

```go
// BenchmarkStateSize measures performance impact of state size
func BenchmarkStateSize(b *testing.B) {
    stateSizes := []int{1000, 10000, 100000, 1000000}

    for _, size := range stateSizes {
        b.Run(fmt.Sprintf("StateSize_%d", size), func(b *testing.B) {
            suite := KeeperTestSuite{}
            suite.SetupTest()

            // Pre-populate state
            contractAddr := deployStorageContract(suite)
            populateStorage(suite, contractAddr, size)

            // Measure read performance
            b.Run("RandomReads", func(b *testing.B) {
                vmdb := suite.StateDB()

                b.ResetTimer()
                b.ReportAllocs()

                for i := 0; i < b.N; i++ {
                    slot := common.BigToHash(big.NewInt(int64(i % size)))
                    _ = vmdb.GetState(contractAddr, slot)
                }
            })

            // Measure write performance
            b.Run("RandomWrites", func(b *testing.B) {
                vmdb := suite.StateDB()

                b.ResetTimer()
                b.ReportAllocs()

                for i := 0; i < b.N; i++ {
                    slot := common.BigToHash(big.NewInt(int64(i % size)))
                    value := common.BigToHash(big.NewInt(int64(i)))
                    vmdb.SetState(contractAddr, slot, value)
                }
            })
        })
    }
}
```

---

4. 벤치마크 실행 및 분석

4.1 벤치마크 실행 명령

```bash
# 전체 벤치마크 실행
go test -bench=. -benchmem -count=5 ./tests/integration/x/vm/...

# 특정 벤치마크 실행
go test -bench=BenchmarkStorageReadCache -benchmem -count=5 ./tests/integration/x/vm/

# CPU 프로파일링과 함께 실행
go test -bench=BenchmarkApplyTransaction -benchmem -cpuprofile=cpu.out ./tests/integration/x/vm/

# 메모리 프로파일링과 함께 실행
go test -bench=BenchmarkBatchedCommits -benchmem -memprofile=mem.out ./tests/integration/x/vm/

# 결과를 파일로 저장
go test -bench=. -benchmem -count=5 ./tests/integration/x/vm/ > benchmark_results.txt
```

4.2 벤치마크 비교 분석

```
# benchstat 설치
go install golang.org/x/perf/cmd/benchstat@latest

# Before/After 비교
go test -bench=. -benchmem -count=10 ./tests/integration/x/vm/ > before.txt
# (최적화 적용 후)
go test -bench=. -benchmem -count=10 ./tests/integration/x/vm/ > after.txt

# 통계적 비교
benchstat before.txt after.txt
```

4.3 예상 결과 형식

```
goos: linux
goarch: amd64
pkg: github.com/cosmos/evm/tests/integration/x/vm
cpu: Intel(R) Core(TM) i9-12900K

                                    │  before.txt  │             after.txt             │
                                    │    sec/op    │   sec/op     vs base              │
StorageReadCacheHit-24                 485.2n ± 2%   48.3n ± 1%  -90.05% (p=0.000 n=10)
StorageReadCacheMiss-24                1.234µ ± 3%   1.201µ ± 2%  -2.67% (p=0.043 n=10)
BatchedCommits/BatchSize_100-24        45.67µ ± 2%   12.34µ ± 1%  -72.97% (p=0.000 n=10)
JournalAppendPreallocated-24           234.5n ± 2%   156.7n ± 1%  -33.18% (p=0.000 n=10)
ApplyTransaction-24                    1.567ms ± 3%  1.234ms ± 2% -21.25% (p=0.000 n=10)
TokenTransfer-24                       2.345ms ± 4%  1.890ms ± 3% -19.40% (p=0.000 n=10)

                                    │  before.txt  │              after.txt               │
                                    │     B/op     │     B/op      vs base                │
StorageReadCacheHit-24                  256.0 ± 0%     48.0 ± 0%   -81.25% (p=0.000 n=10)
BatchedCommits/BatchSize_100-24         45.67k ± 1%   12.34k ± 0%  -72.98% (p=0.000 n=10)
JournalAppendPreallocated-24            8.234k ± 0%   2.456k ± 0%  -70.17% (p=0.000 n=10)

                                    │ before.txt │             after.txt              │
                                    │ allocs/op  │  allocs/op   vs base               │
StorageReadCacheHit-24                  5.000 ± 0%   1.000 ± 0%  -80.00% (p=0.000 n=10)
BatchedCommits/BatchSize_100-24         1.234k ± 0%   0.123k ± 0% -90.03% (p=0.000 n=10)
JournalAppendPreallocated-24              167 ± 0%      45 ± 0%  -73.05% (p=0.000 n=10)
```

5. 성능 목표 및 검증 기준

5.1 최적화별 성능 목표

최적화	측정 항목	현재 예상	목표	검증 벤치마크
Storage Read Cache	캐시 히트 읽기 시간	~500ns	<100ns	`BenchmarkStorageReadCacheHit`
	캐시 히트율 (DeFi)	N/A	>80%	`BenchmarkUniswapSwap`
Batched Commits	100개 상태 커밋	~50μs	<15μs	`BenchmarkBatchedCommits`
	메모리 할당	~50KB	<15KB	동일 (B/op)
Journal Optimization	1000개 엔트리 추가	~250ns	<170ns	`BenchmarkJournalAppendPreallocated`
	메모리 할당	~8KB	<3KB	동일 (B/op)
종합	ApplyTransaction	~1.5ms	<1.2ms	`BenchmarkApplyTransaction`
	TokenTransfer	~2.3ms	<1.9ms	`BenchmarkTokenTransfer`
	블록 빌드 (100 tx)	TBD	-20%	`BenchmarkBlockBuilding`

5.2 회귀 방지 기준

```go
// regression_test.go
package vm

import (
    "testing"
)

// 성능 회귀 임계값 정의
const (
    MaxApplyTransactionTime = 2 * time.Millisecond
    MaxStorageReadTime      = 1 * time.Microsecond
    MaxBatchCommitTime      = 20 * time.Microsecond
    MaxMemoryPerTx          = 100 * 1024 // 100KB
)

func TestPerformanceRegression(t *testing.T) {
    t.Run("ApplyTransaction", func(t *testing.T) {
        result := testing.Benchmark(BenchmarkApplyTransaction)
        nsPerOp := float64(result.NsPerOp())

        if time.Duration(nsPerOp) > MaxApplyTransactionTime {
            t.Errorf("Performance regression: ApplyTransaction took %v, max allowed %v",
                time.Duration(nsPerOp), MaxApplyTransactionTime)
        }
    })

    t.Run("StorageRead", func(t *testing.T) {
        result := testing.Benchmark(BenchmarkStorageReadCacheHit)
        nsPerOp := float64(result.NsPerOp())

        if time.Duration(nsPerOp) > MaxStorageReadTime {
            t.Errorf("Performance regression: StorageRead took %v, max allowed %v",
                time.Duration(nsPerOp), MaxStorageReadTime)
        }
    })

    t.Run("MemoryUsage", func(t *testing.T) {
        result := testing.Benchmark(BenchmarkApplyTransaction)
        bytesPerOp := result.MemBytes

        if bytesPerOp > MaxMemoryPerTx {
            t.Errorf("Memory regression: %d bytes/op, max allowed %d",
                bytesPerOp, MaxMemoryPerTx)
        }
    })
}
```

6. CI/CD 통합

6.1 GitHub Actions 워크플로우

```yaml
# .github/workflows/benchmark.yml
name: Benchmark

on:
  pull_request:
    paths:
      - 'x/vm/**'
      - 'tests/integration/x/vm/**'
  push:
    branches: [main]

jobs:
  benchmark:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.21'

      - name: Run benchmarks
        run: |
          go test -bench=. -benchmem -count=5 \
            ./tests/integration/x/vm/... > benchmark.txt

      - name: Compare with baseline
        if: github.event_name == 'pull_request'
        run: |
          git fetch origin main
          git checkout origin/main
          go test -bench=. -benchmem -count=5 \
            ./tests/integration/x/vm/... > baseline.txt
          git checkout -

          go install golang.org/x/perf/cmd/benchstat@latest
          benchstat baseline.txt benchmark.txt > comparison.txt

          # Check for regressions
          if grep -q "+[0-9]*\.[0-9]*%" comparison.txt; then
            echo "Performance regression detected!"
            cat comparison.txt
            exit 1
          fi

      - name: Upload results
        uses: actions/upload-artifact@v4
        with:
          name: benchmark-results
          path: |
            benchmark.txt
            comparison.txt
```

6.2 로컬 벤치마크 스크립트

```bash
#!/bin/bash
# scripts/run_benchmarks.sh

set -e

RESULTS_DIR="benchmark_results"
mkdir -p $RESULTS_DIR

echo "Running StateDB benchmarks..."
go test -bench=BenchmarkStateDB -benchmem -count=5 \
    ./tests/integration/x/vm/ > $RESULTS_DIR/statedb.txt

echo "Running Transaction benchmarks..."
go test -bench=BenchmarkApply -benchmem -count=5 \
    ./tests/integration/x/vm/ > $RESULTS_DIR/transaction.txt

echo "Running Contract benchmarks..."
go test -bench=BenchmarkToken -benchmem -count=5 \
    ./tests/integration/x/vm/ > $RESULTS_DIR/contract.txt

echo "Running Optimization benchmarks..."
go test -bench="BenchmarkStorageReadCache|BenchmarkBatched|BenchmarkJournal" \
    -benchmem -count=5 ./tests/integration/x/vm/ > $RESULTS_DIR/optimization.txt

echo "Generating summary..."
cat $RESULTS_DIR/*.txt | grep -E "^Benchmark" > $RESULTS_DIR/summary.txt

echo "Done! Results in $RESULTS_DIR/"
```

---

7. 프로파일링 가이드

7.1 CPU 프로파일링

```bash
# CPU 프로파일 생성
go test -bench=BenchmarkApplyTransaction -cpuprofile=cpu.out \
    -benchtime=30s ./tests/integration/x/vm/

# 프로파일 분석
go tool pprof -http=:8080 cpu.out
```

주요 분석 포인트:

- `GetState` 호출 빈도
- `KVStore.Get/Set` 시간
- `journal.append` 오버헤드
- 직렬화/역직렬화 시간

7.2 메모리 프로파일링

```
# 메모리 프로파일 생성
go test -bench=BenchmarkTokenTransfer -memprofile=mem.out \
    -benchtime=30s ./tests/integration/x/vm/

# 힙 할당 분석
go tool pprof -alloc_space -http=:8080 mem.out

# 현재 사용 중인 메모리 분석
go tool pprof -inuse_space -http=:8080 mem.out
```

주요 분석 포인트:

- `stateObject` 할당
- `journal.entries` 슬라이스 성장
- `Storage` 맵 할당
- 임시 바이트 슬라이스

7.3 트레이스 분석

```
# 실행 트레이스 생성
go test -bench=BenchmarkBlockBuilding -trace=trace.out \
    -benchtime=10s ./tests/integration/x/vm/

# 트레이스 뷰어
go tool trace trace.out
```

주요 분석 포인트:

- 고루틴 스케줄링
- GC 일시 중지 시간
- 블록킹 연산
- 병렬 처리 기회

---

8. 결론

8.1 핵심 벤치마크 우선순위

1. **Storage Read Cache**: 가장 큰 성능 개선 기대
   - `BenchmarkStorageReadCacheHit` - 90% 이상 개선 목표
   - `BenchmarkBlockLevelCacheSharing` - 교차 트랜잭션 효과
2. **Batched Commits**: 두 번째로 큰 영향
   - `BenchmarkBatchedCommits` - 70% 이상 개선 목표
   - `BenchmarkBatchSizeImpact` - 최적 배치 크기 결정
3. **Journal Optimization**: 보조적 개선
   - `BenchmarkJournalAppendPreallocated` - 30% 이상 개선 목표
   - `BenchmarkSnapshotRevert` - 중첩 스냅샷 처리
4. **통합 시나리오**: 전체 효과 검증
   - `BenchmarkUniswapSwap` - 실제 DeFi 워크로드
   - `BenchmarkBlockBuilding` - 블록 생성 전체 시간

8.2 다음 단계

1. **Week 1**: 기존 벤치마크 기준선 측정
2. **Week 2-3**: 각 최적화 구현 및 개별 벤치마크
3. **Week 4**: 통합 벤치마크 및 회귀 테스트
4. **Week 5**: 프로파일링 기반 추가 최적화
5. **Week 6**: CI/CD 통합 및 문서화

참고 자료

실제 코드 위치

- StateDB 벤치마크: tests/integration/x/vm/benchmark_statedb.go
- 트랜잭션 벤치마크: tests/integration/x/vm/state_transition_benchmark.go
- 컨트랙트 벤치마크: tests/integration/x/vm/test_benchmark.go
- 스토어 벤치마크: x/vm/store/snapshotmulti/store_bench_test.go

Go 벤치마크 도구

- testing 패키지
- benchstat
- pprof
- trace

문서 버전: 1.0 **작성일**: 2025-01 **작성자**: B-Harvest Core Developer Candidate **페이지 수**: ~25 페이지